

Lab 2 Network Scanning & Enumeration

— Build a Port Scanner in Python

Demonstration

Scan an authorized target

```
PS D:\FSCT\FSCT-8561\Labs\FSCT8561-Labs-Kuangda-Zhu\Lab2> python .\scanner.py
Enter target host: localhost
Enter start port: 20
Enter end port: 21

Target: 127.0.0.1
Scanning TCP ports 20-21...

PORT      STATE      SERVICE

Scan complete.
No open port found.
```

As shown in the image, I successfully scanned localhost (127.0.0.1).

Discovery of known open port

```
PS D:\FSCT\FSCT-8561\Labs\FSCT8561-Labs-Kuangda-Zhu\Lab2> python .\scanner.py
Enter target host: localhost
Enter start port: 30000
Enter end port: 30005

Target: 127.0.0.1
Scanning TCP ports 30000-30005...

PORT      STATE      SERVICE
30000     open      unknown
30001     open      unknown
30002     open      unknown

Scan complete.
3 open port(s) found.
```

As shown in the image, I successfully scanned three open ports on localhost (using server.py from Lab 1 to create three local open ports).

Port range

```
PS D:\FSCT\FSCT-8561\Labs\FSCT8561-Labs-Kuangda-Zhu\Lab2> python .\scanner.py
Enter target host: localhost
Enter start port: 30000
Enter end port: 30005

Target: 127.0.0.1
Scanning TCP ports 30000-30005...

PORT      STATE      SERVICE
30000     open      unknown
30001     open      unknown
30002     open      unknown

Scan complete.
3 open port(s) found.
```

As shown in the image, I can freely choose the port range. (Like 30000-30005)

Robustness tests

A known closed/not-open port

```
PS D:\FSCT\FSCT-8561\Labs\FSCT8561-Labs-Kuangda-Zhu\Lab2> python .\scanner.py
Enter target host: localhost
Enter start port: 443
Enter end port: 443

Target: 127.0.0.1
Scanning TCP ports 443-443...

PORT      STATE      SERVICE

Scan complete.
No open port found.
```

As shown in the image, closed ports are not displayed, and the user is prompted that no open ports were found.

Invalid hostname

```
PS D:\FSCT\FSCT-8561\Labs\FSCT8561-Labs-Kuangda-Zhu\Lab2> python .\scanner.py
Enter target host: hello
Unable to resolve target, please input a valid host.
Enter target host: 
```

As shown in the image, if an invalid hostname is entered, the system will prompt an error until you enter a valid hostname.

Non-numeric port input

```
PS D:\FSCT\FSC-8561\Labs\FSC8561-Labs-Kuangda-Zhu\Lab2> python .\scanner.py
Enter target host: localhost
Enter start port: ab123
Invalid input
Enter start port: 
```

As shown in the image, when a non-numeric port is entered, an error will pop up and ask you to re-enter it.

Port below 1

```
Enter start port: 0
Start port must be 0-65535.
Enter start port: -100
Start port must be 0-65535.
Enter start port: 
```

As shown in the image, if you enter a number less than 0, an error will pop up prompting you for the correct port range.

Port above 65535

```
Enter start port: 99999
Start port must be 0-65535.
Enter start port: 
```

As shown in the image, if you enter a number greater than 65535, an error will pop up prompting you for the correct port range.

Start port greater than end port

```
PS D:\FSCT\FSC-8561\Labs\FSC8561-Labs-Kuangda-Zhu\Lab2> python .\scanner.py
Enter target host: localhost
Enter start port: 50
Enter end port: 49
The end port should be bigger or equal than the start port.
Enter end port: 
```

As shown in the image, if the end port you enter is smaller than the start port, it will prompt you to re-enter.

No open ports in the selected range

```
PS D:\FSCT\FSCT-8561\Labs\FSCT8561-Labs-Kuangda-Zhu\Lab2> python .\scanner.py
Enter target host: localhost
Enter start port: 20
Enter end port: 25

Target: 127.0.0.1
Scanning TCP ports 20-25...

PORT      STATE      SERVICE

Scan complete.
No open port found.
PS D:\FSCT\FSCT-8561\Labs\FSCT8561-Labs-Kuangda-Zhu\Lab2> 
```

As shown in the image, when scanning a port range where no services are running (such as ports 20-25), the scan completes successfully and prompts that no open port was found.

Python-nmap version and comparison

```
PS D:\FSCT\FSCT-8561\Labs\FSCT8561-Labs-Kuangda-Zhu\Lab2> python .\nmap_scanner.py
Enter target host: localhost
Enter start port: 30000
Enter end port: 30005

Target: 127.0.0.1

PORT      STATE      SERVICE
30000     open      ndmps
30001     open      pago-services1
30002     open      pago-services2
30003     closed    amicon-fpsu-ra
30004     closed    amicon-fpsu-s
30005     closed

Scan complete.
PS D:\FSCT\FSCT-8561\Labs\FSCT8561-Labs-Kuangda-Zhu\Lab2> 
```

As shown in the image, the python-nmap version successfully scanned ports 30000–30005.

Comparison

1. Did Nmap identify the same open ports as your Python scanner?

Yes, Nmap identified the exact same open ports as the custom Python scanner.

2. What additional information did Nmap provide?

Nmap provided more detailed service names (such as specific service identifiers) and the exact state of every port scanned.

3. Did Nmap distinguish states that your scanner could not?

Yes, Nmap clearly distinguished closed port states, whereas the basic Python socket scanner primarily focuses on locating open ports.

4. Why is your Python scanner useful for learning even though Nmap is more capable?

Because Nmap's functions are fully encapsulated, writing our own scanner helps us better understand the underlying working logic of network scanning.

Questions

1. Which version required more code?

The raw socket scanner.

2. Which version provided more information?

The python-nmap version.

3. What information did Nmap provide that your raw socket scanner did not?

Detailed service names and precise port states like

4. What work is python-nmap hiding from the programmer?

Low-level socket creation and connection handling.

5. Why was it useful to build the raw socket version before using python-nmap?

To understand the underlying working logic of network scanning.

6. If you were developing a real security application, which approach would you choose and why?

Nmap, because it is more efficient.

Security Reflection

1. What information can an attacker learn from port scanning?

Open ports and exposed running services,

2. Why can an open port increase a system's attack surface?

It provides potential entry for attackers to target.

3. Does an open port automatically mean that a vulnerability exists? Explain.

No. Open port just means a service is listening. It's still decided by the security system itself.

4. Why should unnecessary network services be disabled or restricted?

To reduce the attack surface and minimize potential entry.

5. What is the difference between an open port and a closed port?

An open port accepts connections (service listening), while a closed port rejects them (no service).

6. Why might a firewall make scan results difficult to interpret?

Firewalls filter traffic, and make ports show up as "filtered" instead of clearly open or closed.

7. Why can our simple TCP Connect scanner not reliably identify every filtered port?

Because it might be blocked by the firewalls.

8. How could defenders detect or limit scanning activity?

Using firewalls to filter those activities.

9. Why should defenders scan their own systems?

To find and fix vulnerabilities before attackers can exploit them.

10. Why is authorization important before performing network scanning?

To ensure legal compliance and avoid unauthorized system disruption.

Security Analysis

In this experiment, I was authorized to scan my own local machine (localhost / 127.0.0.1). Although I could technically attempt to scan external public websites such as Google or BCIT, doing so without explicit permission would violate safety and legal regulations, and likely trigger immediate security blocks. Therefore, I chose a safe and compliant approach by testing exclusively within my local environment.

For the experiment, I tested the port range from 30000 to 30005 to accurately evaluate the scanner's ability to distinguish between open and closed ports. Before running the

scans, I utilized sample code from Lab 1 to spin up local servers on ports 30000, 30001, and 30002. As a result, both my custom Python scanner and Nmap successfully detected these three open ports within the targeted range, while the remaining ports (30003 to 30005) were correctly identified as closed. While my custom scanner.py could only verify the open/closed status, Nmap additionally provided recognizable service types for those ports.

Leaving unnecessary services exposed increases potential security risks, as any active port expands the attack surface, providing attackers with potential entry points to exploit vulnerabilities or launch unauthorized access attempts.

Comparing the two tools, the custom Python socket scanner and Nmap function similarly in principle, but Nmap is far more advanced and convenient because it encapsulates complex operations and features a much larger library to identify diverse services.

To defend against these risks and reduce unnecessary exposure, several measures can be implemented:

1. **Firewalls:** Configuring firewalls can hide true port states, making ports appear "filtered" instead of openly exposed, which increases the difficulty for attackers.
2. **Disable Unused Services:** Regularly turning off or disabling unnecessary network services to minimize the attack surface.
3. **Regular Security Audits:** Conducting routine internal port scans to detect and close unauthorized or forgotten listening ports before malicious actors can find them.

Github

<https://github.com/bnwzc/FSCT8561-Labs-Kuangda-Zhu>